```
std::atomic<data_to_reclaim*> nodes_to_reclaim;
void add_to_reclaim_list(data_to_reclaim* node)     <-3
{
    node->next=nodes_to_reclaim.load();
    while(!nodes_to_reclaim.compare_exchange_weak(node->next,node));
}
template<typename T>
void reclaim_later(T* data)     <-4
{
    add_to_reclaim_list(new data_to_reclaim(data));     <-5
}
void delete_nodes_with_no_hazards()
{
    data_to_reclaim* current=nodes_to_reclaim.exchange(nullptr);     <-6
    while(current)
    {
        data_to_reclaim* const next=current->next;
        if(!outstanding_hazard_pointers_for(current->data))     <-7
        {
            delete current;     <-8
        }
        else
        {
            add_to_reclaim_list(current);     <-9
        }
        current=next;
    }
}
```

First off, I expect you've spotted that `reclaim_later()` is a function template rather than a plain function ❹. This is because hazard pointers are a general-purpose utility, so you don't want to tie yourselves to stack nodes. You've been using `std::atomic<void*>` for storing the pointers already. You therefore need to handle any pointer type, but you can't use `void*` because you want to delete the data items when you can, and `delete` requires the real type of the pointer. The constructor of `data_to_reclaim` handles that nicely, as you'll see in a minute: `reclaim_later()` just creates a new instance of `data_to_reclaim` for your pointer and adds it to the reclaim list ❺.
`add_to_reclaim_list()` itself ❸ is just a simple `compare_exchange_weak()` loop on the list head like you've seen before.